

ESP32-S3 Voice Assistant Project — Technical Summary

Project Goal

Develop a controllable and extensible voice-assistant platform using an ESP32-S3 N16R8 board, OLED display, I2S microphone, WiFi networking, and server-side AI processing.

The long-term aim is not merely to run a prebuilt chatbot firmware, but to understand and control the entire signal chain:

- audio capture
- speech detection
- recording logic
- network transmission
- transcription
- AI response generation
- local device feedback

The project is moving toward a modular architecture where the ESP32 acts as a dedicated audio + interface front-end, while heavier AI tasks run externally.

Hardware Platform

Main Controller

- ESP32-S3 N16R8
- 16MB Flash
- 8MB PSRAM

This configuration provides substantially more flexibility than lower-end ESP32 boards because PSRAM allows:

- larger audio buffers
 - simultaneous WiFi + audio processing
 - more stable memory allocation
 - future support for lightweight on-device AI components
-

Connected Hardware

OLED Display

- 0.96" I2C OLED
- SSD1306 compatible

- Address: `0x3C`

Confirmed working.

I2S Microphone

- INMP441 digital microphone

Confirmed working.

Audio / Amplifier

External speaker amplifier module connected for future audio output and response playback.

System Architecture Direction

Initial Assumption

The original approach was based around prebuilt firmware systems similar to those distributed through ecosystems such as TanDev / XiaoZhi.

These systems generally:

- provide precompiled `.bin` firmware
- hide board-level GPIO mappings
- assume specific hardware layouts
- abstract most implementation details away from the user

This creates convenience for quick demonstrations but introduces major limitations when hardware differs from expected reference designs.

Key Discovery

The project gradually shifted from:

“flash-and-use appliance firmware”

into:

“fully understood and controllable AI device architecture”

This changed the engineering direction substantially.

Difficulties Already Overcome

1. OLED Recovery and I2C Diagnosis

Early testing produced:

- blank displays
- frozen images
- apparent OLED failure
- inconsistent firmware behavior

Root causes discovered:

- SDA/SCL lines reversed
- incorrect assumptions about GPIO mappings
- firmware expecting different board profiles

An I2C scanner confirmed:

- OLED address `0x3C`
- OLED hardware functional
- ESP32 I2C communication operational

The OLED subsystem is now stable.

2. Board Profile and Firmware Mapping Issues

A major obstacle was discovering that many ESP32-S3 chatbot firmware systems do not expose their internal GPIO mappings clearly.

The firmware selected through flashing websites often contains:

- hidden board-specific pin assignments
- fixed audio routing assumptions
- fixed display configurations

This caused repeated symptoms such as:

- working WiFi but dead display
- microphone instability
- conflicting GPIO usage
- partially functioning systems

This was not initially obvious because the systems present themselves as universal firmware.

3. I2S Microphone Validation

The INMP441 microphone subsystem was eventually isolated and tested directly.

A dedicated signal-analysis sketch was developed to:

- inspect raw audio values
- measure peak amplitude
- measure average room noise
- determine usable speech thresholds

Observed values:

Silence / Room Noise

Typical range:

- ~100–300

Speech Peaks

Typical range:

- ~7000–20000+

This confirmed:

- the microphone hardware is functional
- I2S configuration is valid
- speech detection is practical
- clean threshold separation exists

This was a major milestone because it eliminated uncertainty around audio hardware.

4. Whisper Integration Problems

Initial attempts used:

- continuous streaming audio
- fragmented HTTP POST requests
- small audio chunks
- incomplete utterances

This caused:

- random transcriptions
- silence detections
- isolated words
- unstable behavior

The issue was not Whisper itself.

The real issue was architectural:

Whisper expects clean speech segments rather than unstable micro-fragment streams.

This realization changed the project direction.

Current Design Philosophy

The system is now moving toward a more deterministic architecture:

```
ESP32-S3
↓
voice detection
↓
record full speech segment
↓
send clean audio block
↓
server-side Whisper transcription
↓
AI response generation
↓
ESP32 display/audio feedback
```

This replaces unstable continuous streaming with controlled audio segmentation.

Current Development Status

Confirmed Working

- ESP32-S3 stable
 - PSRAM detected
 - OLED operational
 - I2C communication operational
 - INMP441 microphone operational
 - WiFi communication operational
 - Flask server communication operational
 - Speech amplitude separation confirmed
-

Current Focus

The project is currently focused on:

Voice Activity Detection (VAD-lite)

The ESP32 will:

- wait in idle state
- detect speech threshold crossing
- begin recording automatically
- stop after sustained silence
- transmit only meaningful speech segments

This is a critical transition point because it stabilizes the entire downstream AI pipeline.

Remaining Obstacles

1. Stable Speech Segmentation

The next challenge is reliably determining:

- speech start
 - speech end
 - silence timeout
 - threshold adaptation across environments
-

2. Clean Audio Packaging

The system must consistently generate:

- valid PCM or WAV data
 - correct sample rate
 - proper buffer lengths
 - clean server uploads
-

3. Whisper Reintegration

Once segmentation is stable:

- Whisper can be reintroduced
 - transcription quality should improve dramatically
 - random word generation should disappear
-

4. Full Conversational Pipeline

Future stages may include:

- GPT integration
 - response memory
 - OLED UI improvements
 - local wake words
 - speaker playback
 - offline command mode
 - hybrid cloud/local inference
-

Comparison Against Typical Prebuilt ESP32 Chatbot Projects

Typical Flash-Only Projects

Most hobbyist projects based on precompiled chatbot firmware:

- prioritize quick demos
- hide implementation details
- assume fixed hardware
- provide little insight into audio architecture
- break easily when hardware differs
- discourage low-level understanding

Users often remain dependent on:

- firmware maintainers
 - undocumented board mappings
 - closed binary builds
-

This Project's Direction

This project is increasingly focused on:

- hardware understanding
- signal analysis
- deterministic system behavior
- modular architecture
- debuggable audio pipelines
- full control of device logic

Rather than treating the ESP32 as a “magic AI gadget”, it is being treated as:

- a measurable signal-processing system

- a controllable embedded platform
 - a configurable edge device for AI interaction
-

Strategic Assessment

The project has already passed several difficult stages that commonly stop ESP32 voice-assistant development:

- OLED/I2C recovery
- GPIO conflict diagnosis
- microphone validation
- threshold analysis
- network communication
- server integration

The remaining work is now less about hardware uncertainty and more about:

- system architecture
- segmentation quality
- AI pipeline stability

This represents a significant transition from experimentation into actual system engineering.

Immediate Next Steps

1. Finalize room-noise and speech thresholds
 2. Implement stable speech-trigger recording
 3. Record complete speech segments locally
 4. Send clean audio blocks to server
 5. Reintroduce Whisper transcription
 6. Validate stable conversational loop
 7. Expand into higher-level assistant functionality
-

Long-Term Vision

The long-term objective is a fully understood voice-interface platform that is:

- modular
- inspectable
- extensible
- not dependent on hidden firmware assumptions
- capable of evolving into more advanced AI interaction systems

This includes the possibility of:

- autonomous edge AI interfaces
- distributed AI devices
- robotic control systems
- wearable or embedded assistants
- local/private AI interaction systems

The project is no longer simply “flashing chatbot firmware.”

It is becoming a controlled embedded AI architecture project.